

Lab 1 – Conteo de Palabras de Wikipedia

CC5212-1 – July 20, 2026

Hoy vamos a contar algunas palabras de los *abstracts* de todos los artículos en español de Wikipedia. Esto con el objetivo de calentar para las siguientes clases.

- Primero hay que descargar algunos datos:

```
– http://aidanhogan.com/teaching/data/wiki/es/es-wiki-abstracts.txt.gz  
– http://aidanhogan.com/teaching/data/wiki/es/es-wiki-abstracts-1k.txt
```

El primer archivo contiene el conjunto de datos completo, el cual cubre 598.385 resúmenes (*abstracts*) de Wikipedia en español (55,2 millones de palabras, 120MB en GZip, 330MB descomprimido). No descomprimas este archivo, ya que trabajaremos con él en el formato en el que lo descargaste. El segundo archivo contiene una muestra, correspondiente a los primeros 1000 resúmenes, la cual puedes abrir, mirar y revisar. Hoy trabajaremos con el archivo más grande. En particular, contaremos la cantidad de ocurrencias de cada palabra, para luego encontrar las palabras más usadas en el español (al menos en los resúmenes de Wikipedia).

- La buena noticia es que, por hoy, el código ya fue escrito por ti. Descárgalo del Material Docente (`gdd-lab01.zip`), descomprímelo y ábrelo como un proyecto Eclipse. Durante el laboratorio ejecutaremos algunas de las clases en este proyecto, tratando de aferrarnos a algunas convenciones. Para averiguar a qué hace referencia cada argumento, simplemente ejecuta la clase sin ningún argumento, o con “-h”. Esto nos ahorra el trabajo que implica tener que explicar cada entrada aquí. Del mismo modo debes reemplazar con algo lo que se vea en gris (o Griego) en un comando. Mientras ejecutas las clases, aprovecha de revisar el código y de ver qué hace.
- Todos los archivos serán codificados en UTF-8. Si observas algún tipo de problema de codificación en tu consola, puede que tengas que cambiar el encoding de la consola de Eclipse. Mira el siguiente link:

```
– https://decoding.wordpress.com/2010/03/18/eclipse-how-to-change-the-console-output-encoding/
```

- Crea un directorio local vacío; este contendrá todos los datos para el laboratorio. De aquí en adelante nos referiremos a este último como `PATH`. Al final del laboratorio (si lo completaste) puedes borrarlo, para liberar espacio en el disco. Trata de usar el mismo directorio en laboratorios futuros, para así mantener todo contenido y limpio. (Pero bueno, es tu notebook.)
- Entonces, lo primero que queremos hacer es contar las apariciones de cada palabra en el archivo grande. Para ello ocuparemos `RunWordCountInMemory`; esta clase utilizará un mapa en memoria, para mantener un registro de la cantidad de veces que aparece cada palabra, para luego ordenarlas por ocurrencia y finalmente retornar las top 100. Ejecutaremos esto sobre el primer archivo. ¿Funcionará? ¿Cuánto demorará en contar las ocurrencias de cada palabra en todos los resúmenes de Wikipedia en español? Inténtalo y ve lo que ocurre:

```
– RunWordCountInMemory -i PATH/es-wiki-abstracts.txt.gz -igz -k 100
```

Ojo que `RunWordCountInMemory` es el nombre de la clase, y lo que sigue son los argumentos de programa que tienes que poner en *Run Configurations* de Eclipse.

¿Cómo fue el tiempo de ejecución?

- No lo suficiente desafiante para la máquina, ¿no? Intentemos con algo más difícil: ahora contaremos los *n*-gramas. Por ejemplo un 2-grama (o bigrama) es una secuencia de dos palabras consecutivas, como “por ejemplo”, un 3-grama es una de tres palabras consecutivas como “qué raro es”. Nuevamente se te entrega código que logra esto:

```
– RunNGramCountInMemory -i PATH/es-wiki-abstracts.txt.gz -igz -k 100 -n 2  
– RunNGramCountInMemory -i PATH/es-wiki-abstracts.txt.gz -igz -k 100 -n 3
```

– `RunNGramCountInMemory -i [PATH]/es-wiki-abstracts.txt.gz -igz -k 100 -n 4`

Sigue aumentando n hasta que ya no funcione. Mantén registro de los tiempos de ejecución al ir aumentando el número de n -gramas que se ejecutan exitosamente.

Si se te empieza a acabar la memoria, el programa va a empezar a ir más lento y, eventualmente, se detendrá (el recolector de basura comienza a trabajar más y más, tratando de mantener al programa vivo). Nota que puedes asignar más *heap* (memoria) a tu programa agregando, por ejemplo, `-Xmx1024M` a los argumentos de la máquina virtual (*VM arguments*); esto entrega 1024 MB de memoria para el *heap*. Puedes intentar agregar más si tu máquina lo permite.

Aumentar la cantidad de memoria debiese permitirte ejecutar el programa para n -gramas mayores, hasta que tu sistema no tenga más que ofrecer. Entrégale cuanta memoria puedas y trata de encontrar el n más pequeño, tal que no puedas correr el programa en la memoria disponible por el sistema.

Entonces, ¿Ahora qué? Tal vez se pueda ocupar el disco duro, pero ¿Cómo?

- (Si prefieres puedes ejecutar lo que sigue sobre el archivo de 1000 líneas: esto te entregará una muestra fácil de analizar y que te permitirá seguir de manera sencilla el proceso. Lo dejaré como opcional. Si lo haces, tendrás que cambiar el nombre del archivo donde corresponda, y además tendrás que omitir las flags `-igz` y `-ogz`, las que indican que tanto entrada como salida corresponden a archivos `gzip`)
- Sea “ ν ” el mínimo n que hizo que lo anterior fallase. Primero hay que ejecutar lo que sigue para escribir los ν -gramas en un archivo, de manera tal que haya solo un ν -grama por fila (quitando la nueva línea):

```
– ExtractNGrams -i [PATH]/es-wiki-abstracts.txt.gz -igz -n  $\nu$ 
–o [PATH]/es-wiki-abstracts- $\nu$ grams.txt.gz -ogz
```

Toma nota del tiempo de ejecución, líneas de ν -gramas escritas y del tamaño en disco del archivo de salida.

Ahora solo tenemos que encontrar cuántas veces aparece cada ν -grama (o cada línea distinta) en el archivo. ¿Cómo podemos hacer esto?

- Ordenaremos los datos, lo que agrupará a las líneas iguales. Como no todos caben en memoria, tendremos que ordenarlos en pequeños lotes (*batches*), escribir cada lote en disco, y luego unir todos los lotes (*merge*). A esto se le llama *external (merge) sort*. Se le llama externo, ya que utiliza memoria externa (en este caso, el disco). Además se le llama *merge sort*, debido a que, en su última etapa, se fusionan todos los archivos correspondientes a los lotes. Nuevamente, se te entrega el código que hace esto.

Para ordenar el archivo obtenido como salida de la última fase, solo debes ejecutar:

```
– ExternalMergeSort -i [PATH]/es-wiki-abstracts- $\nu$ grams.txt.gz -igz
–o [PATH]/es-wiki-abstracts- $\nu$ grams-s.txt.gz -ogz -tmp [PATH]/tmp -b  $\beta$ 
```

Pero, ¿Qué es β ? Es el número de líneas a colocar en cada lote ordenado. Necesitas entregar esto como entrada. Si el valor de β es muy bajo, vas a terminar con muchos archivos correspondientes a los lotes, lo que ralentizará mucho al *merge-sort* que viene. En el caso contrario, puedes quedarte sin memoria (o molestar al recolector de basura). Un buen valor de β dependerá de tu memoria, el largo de las líneas, entre otros factores. Ten en consideración que β debiese cubrir a todos los lotes, por lo que debes asegurarte de que cubra al más grande (¡no al de tamaño promedio!).

Entonces debes encontrar un buen valor de β ; no hay necesidad de ser tan exacto (usa un valor de $\beta > 100.000$ al menos). Primero aumenta el tamaño de tu *heap* (memoria disponible al programa) hasta alrededor de 1 GB. Luego trata de encontrar una aproximación para el valor máximo de β , donde el primer lote funcione bien, pero tal que, digamos, 2β no lo haga. Toma, tal vez, 0.9β para asegurarte de cubrir el peor caso (esto no tiene que ser una ciencia exacta). Una vez tengas un buen valor aproximado para β , ejecuta tu código siguiendo las instrucciones mostradas arriba. Presta atención al número de lotes creados (*batch files*), el tiempo de creación de los lotes ordenados y el que toma fusionarlos.

- Ahora tienes que contar las líneas repetidas/ n -gramas en el archivo de salida de la parte anterior. Como el archivo está ordenado, esto es fácil: las líneas repetidas serán consecutivas, por lo que solo tenemos que escanear el archivo y contar las líneas repetidas que aparecen juntas. De nuevo, el código está listo:

```
- CountDuplicates -i [PATH]/es-wiki-abstracts- $\nu$ grams-s.txt.gz -igz
-o [PATH]/es-wiki-abstracts- $\nu$ grams-c.txt.gz -ogz
```

Mantén un ojo en el tiempo de ejecución.

Esto tendrá como salida un archivo como:

```
...
0000000009 que rara es
0000000007 que raro es
0000000002 que rara esta
0000000003 que raro este
...
```

Las líneas están ordenadas alfabéticamente, basadas en el primer ordenamiento, sin embargo, nos gustaría encontrar las líneas más frecuentes (las que tienen contadores más altos). Nota los ceros al inicio del contador: estos se agregan para así poder re-usar el primer método de ordenamiento para ordenar de manera numérica. Si no estuviesen, entonces “200” podría aparecer antes de “30” en el ordenamiento, pero después de “10”. (Ojo que el inicio de archivo, siendo ordenado y extraído de texto, puede contener ruido. Está bien.)

Entonces, ahora podemos ordenar la salida de la última fase ocupando el ordenamiento, pero en este caso, además, se debe agregar una bandera (*flag*) que indique que se quiere ordenar de manera descendente: queremos que los tengan mayores contadores aparezcan primero. Por lo tanto debemos agregar la bandera ‘-r’; el código hará el resto. Podemos ocupar el mismo valor de β .

```
- ExternalMergeSort -i [PATH]/es-wiki-abstracts- $\nu$ grams-c.txt.gz -igz
-o [PATH]/es-wiki-abstracts- $\nu$ grams-c-s.txt.gz -ogz -tmp [PATH]/tmp/ -b  $\beta$  -r
```

Ahora tenemos los top n -gramas en los datos!

- Pero el archivo es realmente grande, por lo que puede ser difícil abrirlo y ver los resultados. Si estas usando Linux puedes ocupar `zcat [file] | more` para echarle una mirada. Si estás en Windows las cosas son un poco más difíciles, pero se puede usar *Windows Subsystem for Linux (WSL)* si lo tienes, o sino, también se incluye código que puede ayudarte. Ejecuta:

```
- Head -i [PATH]/es-wiki-abstracts- $\nu$ grams-c-s.txt.gz -igz -k 100
-o [PATH]/es-wiki-abstracts- $\nu$ grams-c-s-top100.txt
```

Esto te entregará las primera 100 líneas en el archivo, las cuales puedes abrir.

- Entonces, ¿Cuál de los métodos presentados arriba era más rápido? ¿Cuál era más lento? ¿Cuál era más escalable? ¿Cuáles fueron los beneficios de ocupar el disco duro vs. ocupar la memoria principal? (No hay que entregar los resultados de esta pregunta, pero deberías poder contestarlas. Si no, no dudes en abrir un hilo en el foro del curso.)
- ENTREGA por U-Cursos los top-100 resultados obtenidos del conteo de ν -grams.